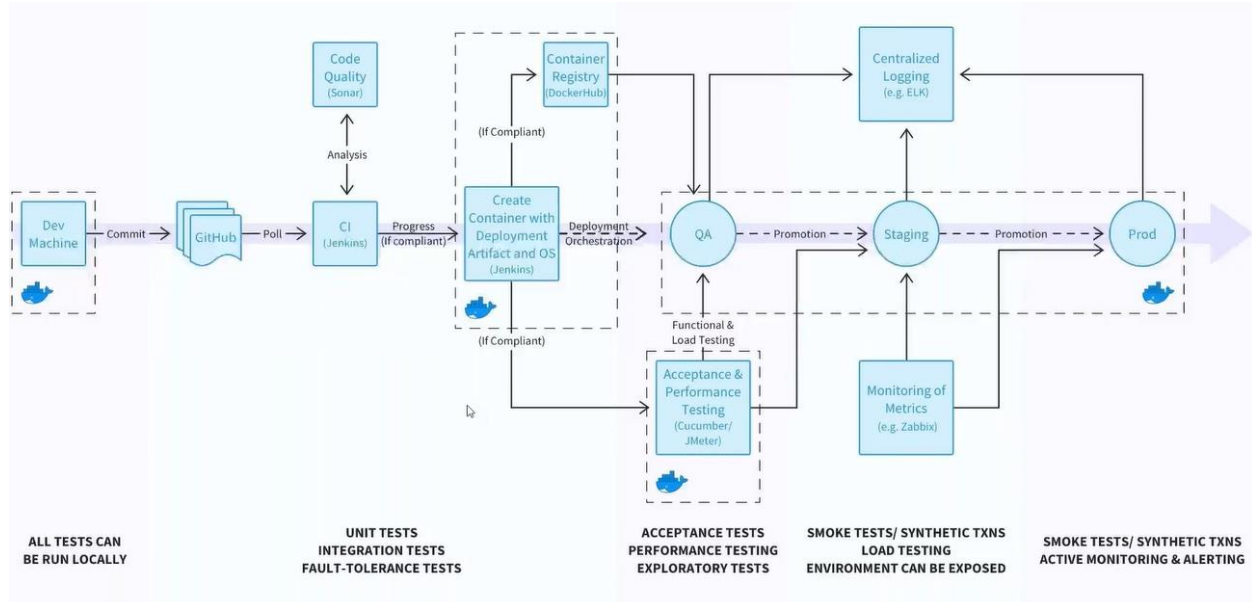
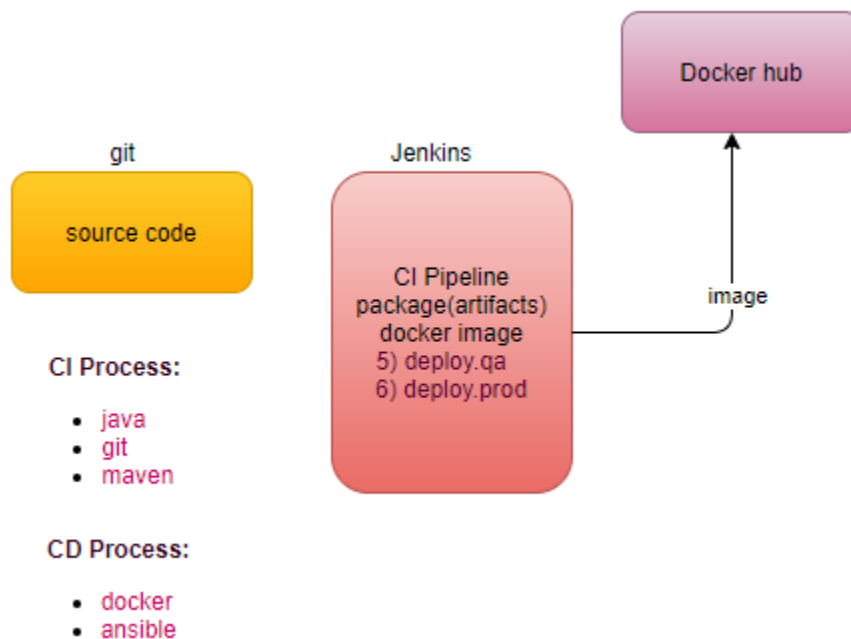




The output of CI => artifact (.war).

How to create docker image with the .war, and how to push it to the dockerhub

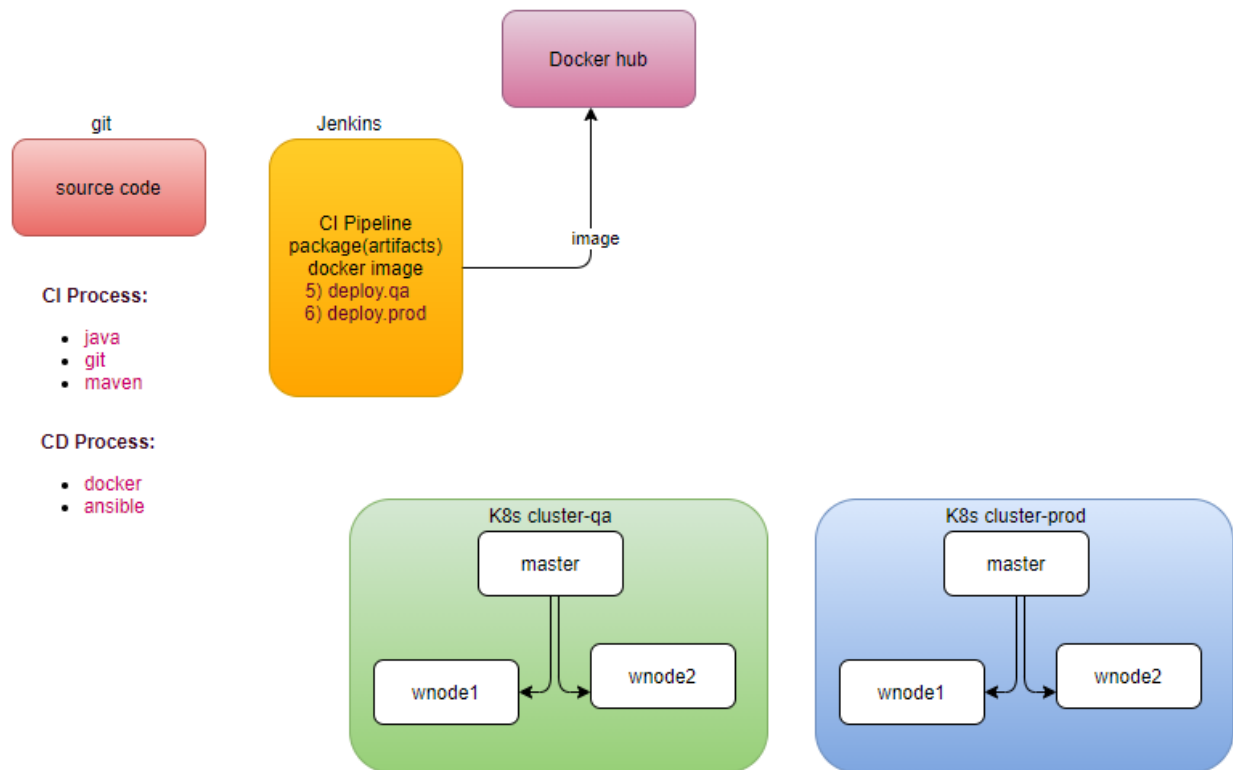


Continuous Delivery Process

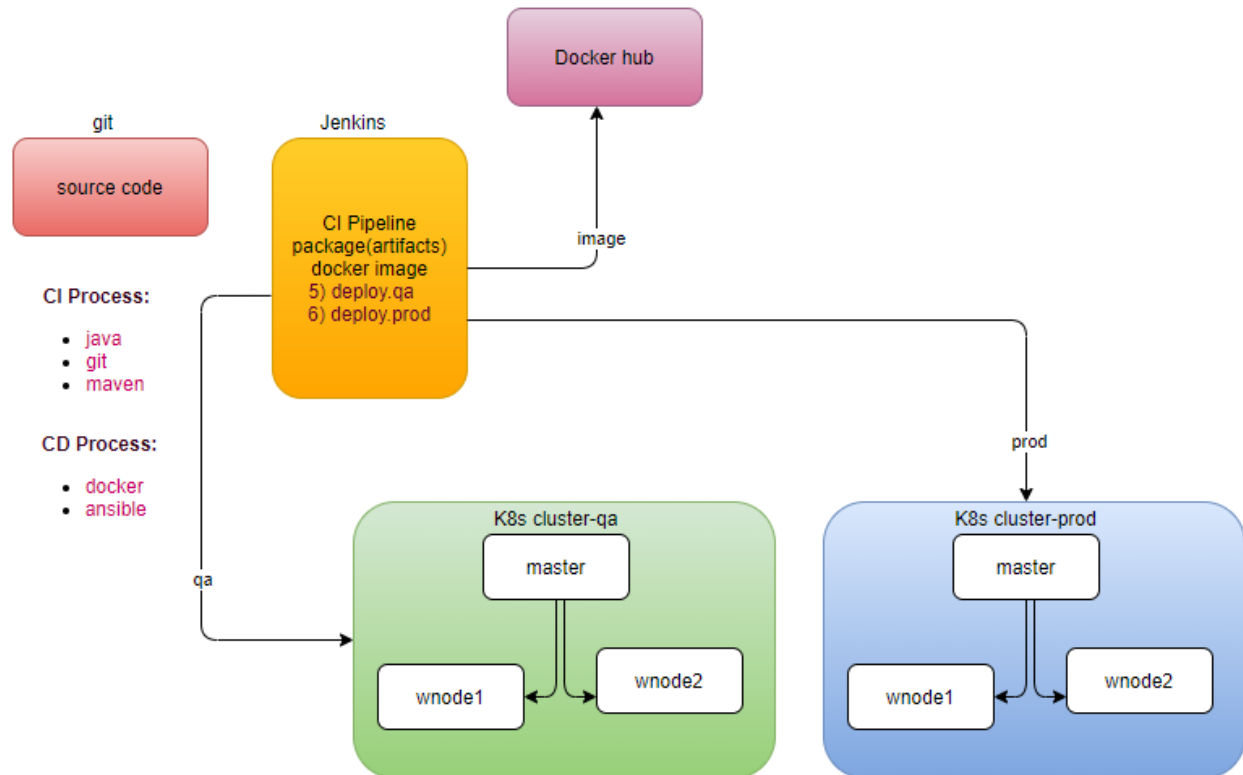
Integrate with docker -> create image

Integrate with ansible-> deploy in different environments (Environment is a group of servers).

pre-requisite-> Before deployment our environment must be ready.



Note: Jenkins's itself doesn't login just trigger ansible to login into k8s master.



build docker image

step1: integrate docker with jenkins

step1.1: Install docker on the jenkins machine, just like we installed maven.

step1.2: **Can jenkins run docker commands?** Permission denied

step1.3: by default, jenkins runs the job as a jenkins user, whereas docker run as user root (access O/S kernel), means jenkins user doesn't have root privileges

```
root@jnode:/var/lib/jenkins/workspace# su - jenkins
jenkins@jnode:~$
jenkins@jnode:~$ docker ps
WARNING: Error loading config file: /var/lib/jenkins/.docker/config.json: open /var/lib/jenkins/.docker/config.json: permission denied
Got permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json: dial unix /var/run/docker.sock: connect: permission denied
jenkins@jnode:~$
```

step1.4: su - jenkins

docker ps # permission denied

sudo docker ps # no sudoers

sudo docker ps # ask for password

```

jenkins@jnode:~$ sudo docker ps
[sudo] password for jenkins:
jenkins@jnode:~$

```

step1.5: Give a Jenkins user sudoers privilege, or add jenkins user in the docker group.

/etc/sudoers

```

root@jnode:/var/lib/jenkins/workspace# cat /etc/sudoers
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults    env_reset
Defaults    mail_badpass
Defaults    secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"
#
# Host alias specification
#
# User alias specification
#
# Cmnd alias specification
#
# User privilege specification
root    ALL=(ALL:ALL) ALL
#
# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL
#
# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL
#
# See sudoers(5) for more information on "#include" directives:
#includedir /etc/sudoers.d
devops  ALL=(ALL) NOPASSWD: ALL
root@jnode:/var/lib/jenkins/workspace#

```

visudoers

```

#includedir /etc/sudoers.d
devops ALL=(ALL) NOPASSWD: ALL
jenkins ALL=(ALL) NOPASSWD: ALL
~

```

service jenkins restart # service command always execute as a root user

now jenkins user can execute docker command as sudo

```

jenkins@jnode:~$ docker ps
WARNING: Error loading config file: /var/lib/jenkins/.docker/config.json: open /var/lib/jenkins/.docker/cnfig.json: permission denied
Got permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/json: dial unix /var/run/docker.sock: connect: permission denied
jenkins@jnode:~$
jenkins@jnode:~$
jenkins@jnode:~$
jenkins@jnode:~$
jenkins@jnode:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
jenkins@jnode:~$

```

Validate:

re-login in to jenkins console

How to run docker commands from Jenkins's console?

(shell script)

In the package job

The screenshot shows the Jenkins 'Build' configuration page. It has two main sections: 'Invoke top-level Maven targets' and 'Execute shell'. The 'Invoke top-level Maven targets' section has a 'Maven Version' dropdown set to 'maven3.6' and a 'Goals' text field containing 'package'. There is an 'Advanced...' button to the right. The 'Execute shell' section has a 'Command' text area containing two lines: 'sudo docker ps' and 'sudo docker pull tomcat'. At the bottom, there is a link to 'See the list of available environment variables'.

Build

Invoke top-level Maven targets

Maven Version: maven3.6

Goals: package

Advanced...

Execute shell

Command: sudo docker ps
sudo docker pull tomcat

See [the list of available environment variables](#)

How to build an automated process?

Dockerfile

Where docker clones the repository, under the workspace

```
root@jnode:/var/lib/jenkins/workspace# cd package
root@jnode:/var/lib/jenkins/workspace/package# ls -l
total 76
-rw-r--r-- 1 jenkins jenkins 117 Feb  2 07:01 Dockerfile
-rw-r--r-- 1 jenkins jenkins 1692 Feb  2 07:01 Jenkinsfile
-rw-r--r-- 1 jenkins jenkins 30 Feb  2 10:03 README.md
-rw-r--r-- 1 jenkins jenkins 1746 Feb  2 07:01 build.gradle
drwxr-xr-x 2 jenkins jenkins 4096 Feb  2 07:01 deploy
drwxr-xr-x 3 jenkins jenkins 4096 Feb  2 07:01 gradle
-rwxr-xr-x 1 jenkins jenkins 5764 Feb  2 07:01 gradlew
-rw-r--r-- 1 jenkins jenkins 2942 Feb  2 07:01 gradlew.bat
-rw-r--r-- 1 jenkins jenkins 821 Feb  2 07:01 jenkinsfile-k8s
-rw-r--r-- 1 jenkins jenkins 1627 Feb  2 07:01 jenkinsfile-win
-rw-r--r-- 1 jenkins jenkins 20437 Feb  2 07:01 pom.xml
-rw-r--r-- 1 jenkins jenkins 93 Feb  2 07:01 settings.gradle
drwxr-xr-x 4 jenkins jenkins 4096 Feb  2 07:01 src
drwxr-xr-x 10 jenkins jenkins 4096 Feb 13 15:41 target
root@jnode:/var/lib/jenkins/workspace/package#
```

step1: In the package job

Build

Invoke top-level Maven targets

Maven Version

maven3.6

Goals

package

Advanced...

Execute shell

Command

```
cd $WORKSPACE ### cd /var/lib/jenkins/workspace/package
sudo docker ps
sudo docker pull tomcat
```

See [the list of available environment variables](#)

Advanced...

Execute shell

Command

```
cd $WORKSPACE ### cd /var/lib/jenkins/workspace/package
sudo docker build --file Dockerfile --tag lerndevops/samplejavaapp:$BUILD_NUMBER .
sudo docker login -u lerndevops -p
sudo docker push lerndevops/samplejavaapp:$BUILD_NUMBER
```

See [the list of available environment variables](#)

Advanced...

/*

#!/bin/sh

WORKSPACE=/var/lib/jenkins/workspace/package

cd "\$WORKSPACE"

\$WORKSPACE ##cd /var/lib/jenkins/workspace/package

sudo docker build --file Dockerfile --tag
neerajvohra/samplejavaapp:\$BUILD_NUMBER .

sudo docker login -u neerajvohra -p \$Docker_Hub_Pass

sudo docker push neerajvohra/samplejavaapp:\$BUILD_NUMBER */

step1.1: How to encrypt password?

Manage Jenkins-> Manage Credentials # plugin Credentials

The image shows the Jenkins 'Credentials' management interface. The top section displays a table of credentials for the 'Jenkins' store, with columns for 'T', 'P', 'Store', 'Domain', 'ID', and 'Name'. A single credential named 'sonar' is listed with domain '(global)'. Below this, the 'Stores scoped to Jenkins' section shows the 'Jenkins' store with a dropdown for domains, currently set to '(global)'. The bottom section shows the 'Global credentials (unrestricted)' page, which is currently empty. The 'Add Credentials' form is open, showing the 'Kind' dropdown menu with options: 'Username with password', 'GitHub App', 'SSH Username with private key', 'Secret file', 'Secret text', and 'Certificate'. The 'Secret text' option is selected. The form also includes fields for 'ID' (set to 'DOCKER_HUB_PWD') and 'Description' (set to 'DOCKER_HUB_PWD'). An 'OK' button is visible at the bottom of the form.

Credentials

T	P	Store	Domain	ID	Name
		Jenkins	(global)	sonar	sonar

Icon: S M L

Stores scoped to Jenkins

P	Store	Domains
	Jenkins	(global) ▼

Dashboard > Credentials > System > Global credentials (unrestricted)

Back to credential domains

Add Credentials

Global credentials (unrestricted)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
This credential domain is empty. How about adding some credentials?			

Icon: S M L

Kind: Username with password

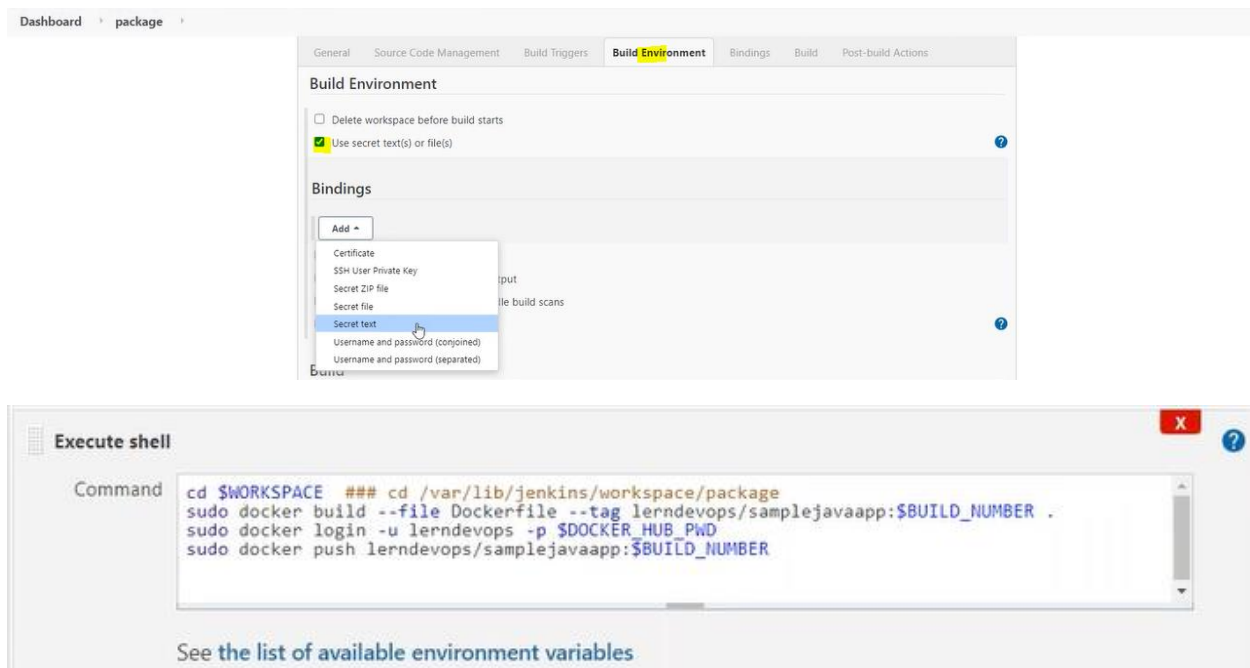
- Username with password
- GitHub App
- SSH Username with private key
- Secret file
- Secret text**
- Certificate

ID: DOCKER_HUB_PWD

Description: DOCKER_HUB_PWD

OK

step1.2: How to get my credentials in my Job



How Integrate Ansible with Jenkins?

step1: setup the environment (k8s cluster)

step2: Jenkin can't log into master servers, will use ansible as a configuration tool to invoke the k8s client, we have to integrate ansible with jenkins

step2.1: If jenkins invoke ansible playbook or modules, install ansible on the same machine in which jenkins server is running.

step2.2: now we can call this machine is as an ansible controller, how controller identify the target servers (k8s master)? **Inventory**

step2.3:

su – devops # switch to devops user

vi myinv



step2.4: Now we need connection also, with username and password or username and sshkey. If we go with username and password then we have to again go with jenkins console.

step2.5: configure ssh keys

step2.5.1: generate ssh for ansible controller

```
ssh-keygen
```

```
ssh-copy-id devops@<ip of target masterqa>
```

```
ssh-copy-id devops@<ip of target masterprod>
```

step2.5.2: validate

```
ssh devops@<ip of target masterqa>
```

```
ssh devops@<ip of target masterprod>
```

```
ansible all -i myinv -m ping
```

step3: create a new job -> freestyle deploy-qa

step3.1: where is my playbook?



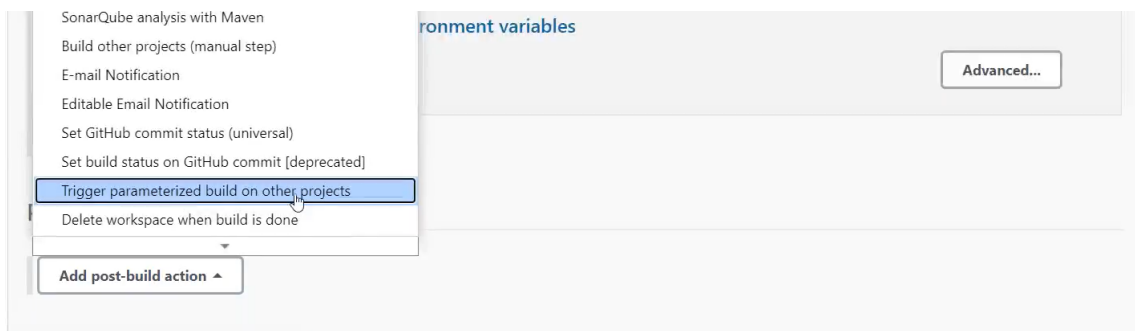
\$BUILD_NUMBER: must be from previous section.

How to pass environment to next job? We have to add **plugin build with parameter** #CREDENTIAL_BINDING

Enabled	Name ↓	Version	Previously installed version	Uninstall
Build With Parameters				
☒	Allows the user to provide parameters for a build in the url (similar to /job/JOBNAME/buildWithParameters), prompting for confirmation before triggering the job.	1.5		<button>Uninstall</button>
Docker plugin				
☒	This plugin integrates Jenkins with Docker	1.2.1		<button>Uninstall</button>
Pipeline				
☒	A suite of plugins that lets you orchestrate automation, simple or complex. See Pipeline as Code with Jenkins for more details.	2.6		<button>Uninstall</button>

Step4: Go to the source job (package)

Add post-build-action-> Trigger parameterized build on other projects.



Post-build Actions

Trigger parameterized build on other projects

Build Triggers

Projects to build

deploy.qa,

Trigger when build is

Stable

Trigger build without parameters

☐

Add Parameters ▾

Boolean parameters

Build on the same node

Current build parameters

Parameters from properties file

Pass-through Git Commit that was built

Predefined parameters

Restrict matrix execution to a subset

Subversion revision

Add trigger...

Add post-build action ▾

Post-build Actions

Trigger parameterized build on other projects

Build Triggers

Projects to build

deploy.qa,

Trigger when build is

Stable

Trigger build without parameters

☐

Predefined parameters

Parameters

package_build_number=\$BUILD_NUMBER

Advanced...

Add Parameters ▾

Step5: Go to the target job (deploy-qa)

The screenshot shows the Jenkins job configuration page for a job named 'deploy-qa'. The 'General' tab is selected, showing various options for build triggers, environment, and build actions. A dropdown menu is open, showing the 'Add Parameter' button and a list of parameter types: Boolean Parameter, Choice Parameter, Credentials Parameter, File Parameter, List Subversion tags (and more), Multi-line String Parameter, Password Parameter, Run Parameter, and String Parameter. The 'Advanced...' button is also visible.

General Source Code Management Build Triggers Build Environment Build Post-build Actions

[Plain text] [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☐ This build requires lockable resources

☒ This project is parameterized ?

☐ Throttle builds

☐ Disable this project

☐ Execute concurrent builds

Add Parameter

- Boolean Parameter
- Choice Parameter
- Credentials Parameter
- File Parameter
- List Subversion tags (and more)
- Multi-line String Parameter
- Password Parameter
- Run Parameter
- String Parameter

[Advanced...](#)

Source Code Man

☐ Discard old builds
☐ GitHub project
☐ This build requires lockable resources
☒ This project is parameterized

String Parameter

Name

package_build_number

Default Value

Description

Build

Execute shell

Command

```
SPACE/deploy
ansible-playbook -i /home/devops/myinv deploy-kube.yml --extra-vars "env=qa build=$package_build_number"
```

L*#!/bin/sh

WORKSPACE=/var/lib/jenkins/workspace/ApplicationDeploy/deploy

cd "\$WORKSPACE"

sudo su devops -c "ansible-playbook -i /home/devops/myinv deploy-kube.yml --extra-vars='env=qa build=\$package_build_number'" *

permission denied, ansible command is running as a jenkins user.

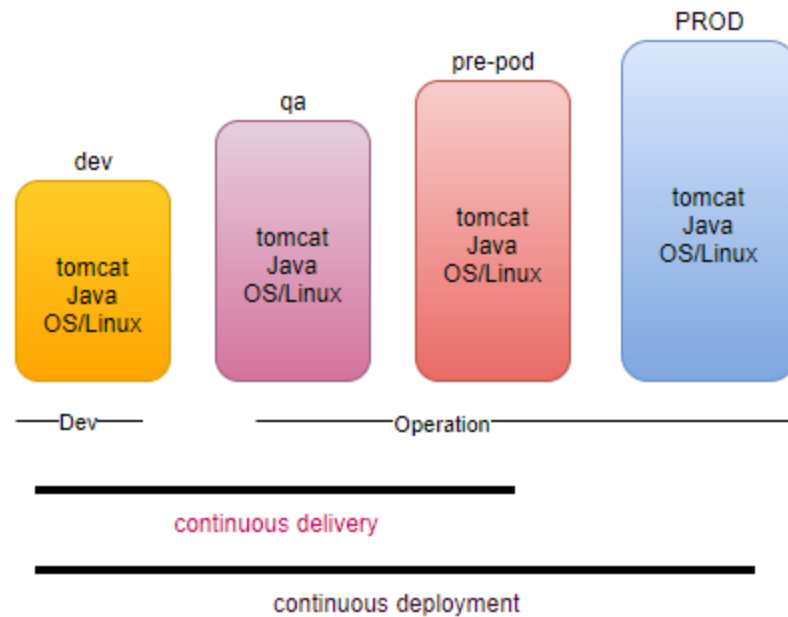
We have shared the ssh key for devops user but not for ansible user.

Build

Execute shell

Command

```
cd $WORKSPACE/deploy
sudo su devops -c "ansible-playbook -i /home/devops/myinv deploy-kube.yml --extra-vars 'env=qa"
```



Continuous delivery is automated, with a single click our application available for QA.



How to create job deploy-prod, pass the variable from deploy-qa

Cloud provided devops tools.

DevOps OnPremise

Git/GitHub
Jenkins (maven)
Ansible/Puppet
Docker
dockerswarm/k8s
Nagios

DevOps on cloud / AWS

code commit
codebuild/codepipeline/codedeploy
Cloud Formation
Docker
ECS / EKS
cloud watch

- DevOps is a mindset, adapt the changes quickly and work on that.
- It's a collaboration with various team. The way you implement, using various tools.
- Deliver your application in the market quicker ASAP.
- Adapting the new tools and technologies.

